

ResNet

📄 Deep Residual Learning for Image Recognition (ResNet)

🔗 Logical Flow

[Deep network의 degradation 문제]

→ [왜 layer가 깊어지면 학습 오차가 커지는가?]: 기존 vanishing gradient로 설명 안 되는 현상 발견

→ [residual를 학습하자]: 최적화 관점의 새 문제 정의

→ [shortcut connection으로 $F(x)+x$ 구현]

: identity mapping을 쉽게 만드는 재구성 아이디어 & 파라미터 없는 shortcut으로 구현

→ [ImageNet/CIFAR-10에서 layer-performance 관계 검증]: depth 152까지 정확도 향상으로 가설 검증

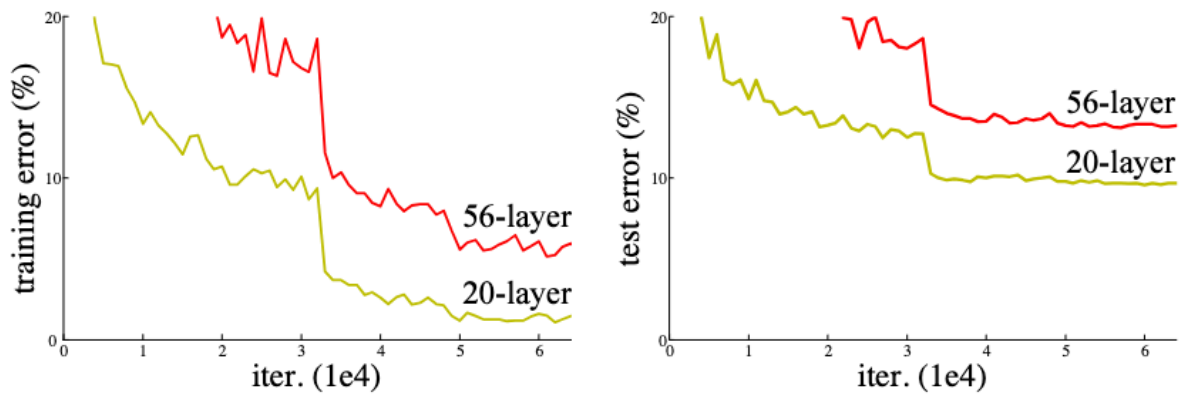
1 Abstract

이 논문은 "네트워크를 residual function(입력과 목표 출력의 차이를 학습하는 함수) 학습 구조로 재구성하면, Deep Neural Network도 쉽게 학습시킬 수 있다"는 것을 핵심 주장으로 합니다. 저자들은 depth 152 짜리 Residual Network(ResNet)를 제안하여 ImageNet에서 3.57%의 top-5 오류율을 달성했습니다. 또한 CIFAR-10에서 depth 1000 이상의 네트워크도 학습 가능함을 보였습니다. 이미지 분류 뿐만 아니라 객체 탐지(COCO에서 28% 상대적 성능 향상) 등 다른 비전 과제에도 일반화되었습니다.

2 Problem Definition (Research Gap)

기존 연구들은 네트워크의 depth가 성능에 결정적으로 중요하다는 것을 보여주었고, VGG(depth 16~19), GoogLeNet(depth 30) 등의 모델들이 ImageNet 최고 성능을 차지하고 있었습니다. 자연스러운 질문은 "더 많은 레이어를 쌓기만 하면 더 좋은 네트워크를 얻을 수 있는가?"였습니다.

과거에는 이 질문에 답하기 어려운 이유가 vanishing/exploding gradient였는데, 이는 normalized initialization과 Batch Normalization 같은 중간 정규화 층으로 상당 부분 해결된 상태였습니다.

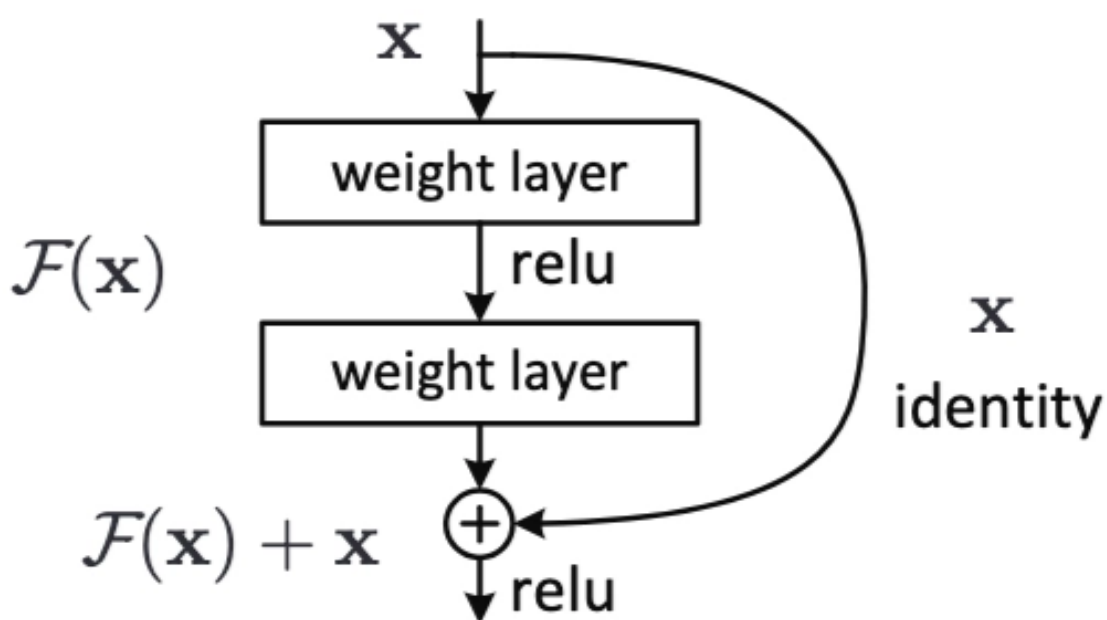


그런데 저자들은 여기서 새로운 현상을 발견합니다: gradient 문제가 해결되어 학습이 정상적으로 시작되는 상황에서도, 네트워크가 일정 깊이를 넘으면 정확도가 saturate 되었다가 오히려 급격히 나빠지는 "degradation 문제"가 나타난다는 것입니다. **중요한 점은 이것이 overfitting 때문이 아니라는 것입니다 — 오히려 training error 자체가 층을 쌓을수록 커 집니다.**

이는 이론적으로 이상한 일입니다. 논리적으로, shallow 모델에 층을 추가하고 그 추가된 층들을 identity mapping으로만 구성해도, 최소한 shallow 모델만큼의 성능은 나와야 합니다. 하지만 실제 SGD 옵티마이저는 이를 찾아내지 못했습니다.

따라서 저자들은 문제를 다음과 같이 재정의합니다: "단순히 층을 쌓는 것이 아니라, solver가 **identity mapping**에 가까운 해를 더 쉽게 찾을 수 있도록 네트워크 구조 자체를 재구성할 수 있는가?"

3 Core Idea



핵심 아이디어는 다음과 같습니다: 몇 개의 층이 원하는 목표 함수 $H(x)$ 를 직접 근사하도록 학습시키는 대신, 그 층들이 잔차 함수 $F(x) := H(x) - x$ 를 근사하도록 만들자는 것입니다. 그러면 원래 함수는 $F(x) + x$ 로 표현됩니다.

Q. Why solvers converge much faster than standard solvers that are unaware of the residual nature of the solutions.

Partial Differential Equation 수치해석 분야에서 **residual** 해를 구하는 것이 **original value**를 직접 구하는 것보다 쉽다는 것이 증명됨

왜 이것이 문제를 해결하는가? 만약 특정 층 블록에서 최적의 해가 identity mapping에 가깝다면, 일반적인 층 구조에서는 solver가 복잡한 비선형 함수들의 조합으로 항등 함수를 재현해야 하는데, 이는 어렵습니다. 반면 잔차 구조에서는 solver가 그저 가중치들을 0에 가깝게 만들기만 하면 $F(x) \approx 0$ 이 되어 자동으로 identity mapping이 됩니다. 즉, **"아무것도 하지 않는 것"을 학습하는 것이 "무언가를 정확히 재현하는 것"보다 훨씬 쉽다는 것이 저자들의 가설입니다.**

이는 기존 shortcut connection 연구(MLP의 auxiliary classifier 등)와는 다릅니다. Highway network는 데이터에 따라 게이트가 열리고 닫히는 파라미터화된 shortcut을 쓰지만, 이 논문의 shortcut은 파라미터가 전혀 없는 순수 identity이며 항상 열려 있습니다. 저자들은 이 단순함 덕분에 depth 100을 훨씬 넘는 깊이에서도 정확도 향상이 가능했다고 대조합니다.

4 Detailed Method

기본 블록: $y = F(x, W_i) + x$ (Eqn. 1). 예를 들어 2개 층으로 구성된 블록이면 $F = W_2 \cdot \sigma(W_1 x)$. 이 shortcut 덧셈은 element-wise addition으로, 추가 파라미터나 연산량을 거의 유발하지 않습니다.

차원이 다를 때: x 와 $F(x)$ 의 차원이 다르면(예: 채널 수가 바뀔 때) 두 가지 옵션을 씁니다 — (A) 부족한 차원을 0으로 채우는 zero-padding shortcut, (B) 1×1 convolution으로 linear projection하는 shortcut. 실험적으로 두 가지 옵션 사이 차이는 작아서, 저자들은 "identity shortcut 자체가 핵심이지 projection이 필수는 아니다"라고 결론 내립니다.

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

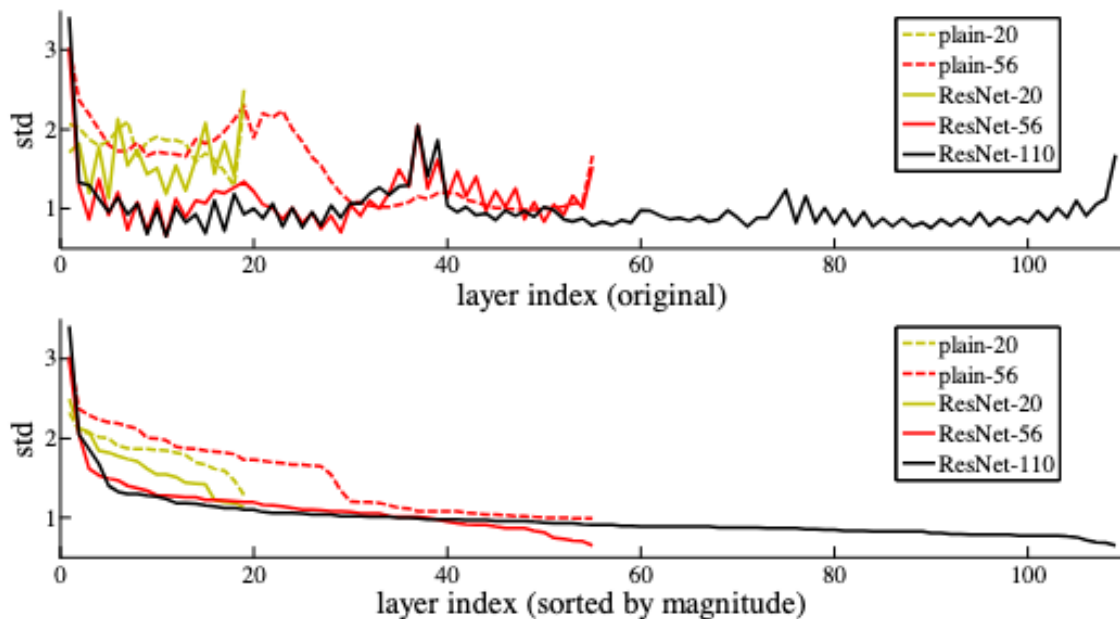
각 구성 요소의 역할을 Core Idea에 매핑하면: shortcut connection = "identity mapping", $F(x)$ = "residual trainer" 역할을 합니다.

5 Experiment

Reason for this setting: Problem Definition에서 제기한 "깊이 자체가 학습을 어렵게 만드는가"라는 질문에 답하기 위해, 저자들은 plain 네트워크와 residual 네트워크를 파라미터 수·깊이·연산량을 동일하게 맞추는 채 직접 비교하는 실험을 설계했습니다. 이는 성능 향상이 "shortcut 유무" 때문임을 명확히 하기 위함입니다.

ImageNet 결과: Layer 18/34 plain 네트워크 비교에서 layer 34이 오히려 높은 오류율을 보여 degradation 문제를 재확인했습니다 (top-1 오류 28.54% vs 27.94%). 반면 layer 34 ResNet은 layer 18 ResNet보다 오류율이 낮아졌고(25.03% vs 27.88%), plain-34 대비 3.5%p 개선되었습니다. 이는 Core Idea의 주장 — "residual 재구성이 깊이에 따른 성능 저하를 해결한다" — 을 직접 뒷받침합니다. 50/101/152층으로 갈수록 계속 정확도가 향상되어(top-5 오류 5.71%까지) "깊을수록 좋다"는 것이 residual 구조 하에서는 성립함을 보였습니다.

Identity vs Projection shortcut 비교: A(zero-padding)/B(일부 projection)/C(전부 projection) 세 옵션이 성능 차이가 미미하다는 결과는, "핵심은 identity shortcut의 존재 자체이지, 추가 파라미터가 아니다"라는 Core Idea를 뒷받침하는 근거로 쓰였습니다.



Layer response 분석: 각 층의 출력 표준편차를 측정해, ResNet의 residual 함수들이 plain 네트워크의 대응 층보다 전반적으로 작은 값을 낸다는 것을 확인했습니다. 이는 "identity mapping에 가까운 해가 최적일 때 residual이 0에 가깝게 학습된다"는 가설을 직접 검증하는 ablation적 성격의 분석입니다.

Q. 그렇다면 논문은 ImageNet에 대해 ideal function은 identity mapping임을 주장하는 것인가?

논문의 가설은 Optimal function이 identity mapping에 가까울 수록 $F(x)$ 는 0에 가깝도록 학습되고, response가 작아야 한다. 이는 실험 결과로 증명되었고, identity mapping이 합리적임을 시사한다.

Q-1. residual 구조로 인해 $F(x)$ 가 작아졌을 가능성은 없는가?

Q-2. 추가적으로 Plain network와 ResNet의 $F(x)$ 를 비교하는데, ResNet의 경우 $F(x) + x$ 를 비교대상으로 설정하는게 합리적이지 않은가?

CIFAR-10, 1000층 이상 실험: ImageNet과 다른 데이터셋에서도 동일한 현상이 재현되어, 이 문제가 특정 데이터셋에 국한된 것이 아님을 뒷받침합니다. 다만 1202층 모델은 110층 모델보다 테스트 성능이 낮았는데, 저자들은 이를 작은 데이터셋에 비해 모델이 과도하게 커서 생기는 과적합으로 해석하며, 이 논문에서는 별도의 정규화 기법(dropout, maxout)을 의도적으로 쓰지 않았다는 한계를 명시했습니다.

💡 한 줄 정리

Deep network의 학습이 어려운 것은 gradient 소실이 아니라 optimization 자체의 문제이며, 층이 identity mapping을 학습하기 쉽도록 잔차 형태 $F(x) + x$ 로 재구성하면 파라미터 증가 없이도 수백 층 규모의 네트워크를 안정적으로, 그리고 더 정확하게 학습시킬 수 있다.

? Questions

We conjecture that the deep plain nets may have exponentially low convergence rates, which impact the reducing of the training error³. The reason for such optimization difficulties will be studied in the future. *p5*

Q. Is this studied?

Li et al. (2018), "Visualizing the Loss Landscape of Neural Nets"

Deep Learning은 Non-convex optimization problem이다. Non-Linear function이 stack되면 loss landscape의 roughness가 증가한다. Gradient가 vanish/explode하지 않아도 landscape가 rough하여 converge가 slow/impossible 했던 것이다. ResNet의 skip connection은 이를 완화한다.

Last, we also note that the 18-layer plain/residual nets are comparably accurate (Table 2), but the 18-layer ResNet converges faster (Fig. 4 right vs. left). *p6*

Q. Why 18-layer ResNet converges faster?

논문에서 별도로 이유를 서술하지 않지만, 앞선 질문에 대한 연구인 Loss landscape의 완만화가 학습 초기의 gradient descent를 가속화했다고 추정할 수 있다.